

Resumen de la Infraestructura Creada

El código despliega una arquitectura web de alta disponibilidad, segura y tolerante a fallos en AWS para albergar una aplicación Apache Tomcat 11 corriendo sobre Java 25 (Early Access), conectada a una base de datos administrada y con almacenamiento compartido.

Componentes de Red (`red.tf` , `variables.tf`)

- **VPC personalizada:** Creación de una red virtual aislada con el bloque CIDR 10.0.0.0/16.
- **Subredes Públicas y Privadas:** Se despliegan 2 subredes públicas y 2 subredes privadas distribuidas en dos Zonas de Disponibilidad distintas (a y b) para asegurar la alta disponibilidad.
- **Internet Gateway (IGW) y NAT Gateway:** Las subredes públicas tienen acceso directo a Internet a través del IGW. Las subredes privadas salen a Internet de forma segura (para descargar paquetes o actualizaciones) a través de un NAT Gateway situado en la primera subred pública.

Capa de Cómputo y Escalado (`aplicacion.tf`)

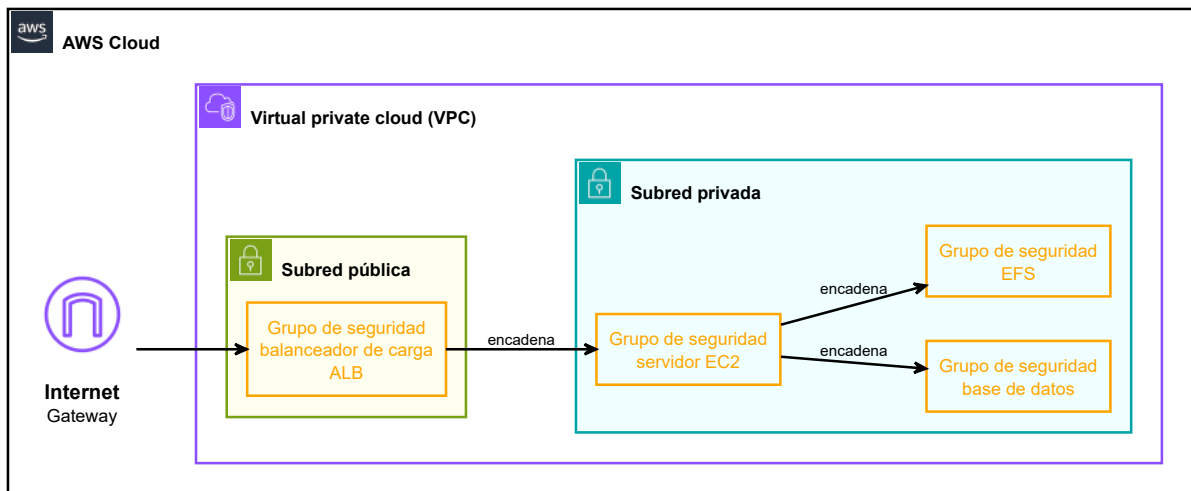
- **Application Load Balancer (ALB):** Desplegado en las subredes públicas para recibir el tráfico externo (HTTP/HTTPS) y distribuirlo eficientemente.
- **Auto Scaling Group (ASG):** Configurado en las subredes privadas (aislado de Internet). Mantiene de forma elástica entre 1 y 2 instancias EC2.
- **Launch Template:** Define cómo se crean las instancias usando Amazon Linux 2023 (t3.micro). Incorpora el script de inicialización ([staging-web.sh](#)) embebido mediante templatefile. En este script se instala Java 25, Tomcat 11 y se configura la aplicación para que lea las credenciales de Secrets Manager en tiempo de arranque.

Capa de Datos y Almacenamiento (`aplicacion.tf`)

- **Amazon RDS (MySQL 8.0):** Una base de datos administrada (db.t3.micro) ubicada exclusivamente en las subredes privadas. Las credenciales de acceso se inyectan dinámicamente y de forma segura extraiéndolas desde AWS Secrets Manager.
- **Amazon EFS (Elastic File System):** Sistema de archivos compartido y persistente en red. Se monta automáticamente en el directorio `/opt/tomcat/webapps/ROOT/uploads` de cada instancia EC2 que levante el ASG, garantizando que si una instancia muere o se escala, los archivos subidos por los usuarios no se pierdan.

Seguridad y Gestión de Secretos (seguridad.tf, aplicacion.tf)

- **AWS Secrets Manager:** Almacena de forma centralizada y encriptada un JSON con datos sensibles: credenciales de la BD, del administrador de Tomcat y el token de firma hmacSHA256.
- **Políticas de Security Groups (Mínimo Privilegio):**
 - El ALB acepta conexiones abiertas a los puertos 80 y 443.
 - Las instancias web (EC2) solo aceptan tráfico en el puerto 80 si este proviene estrictamente del Security Group del ALB.
 - La base de datos (RDS) solo acepta conexiones en el puerto 3306 desde el SG de las EC2 web (e incluye una regla condicional para acceso de administración externa desde la IP configurada en admin_ip) .
 - El EFS solo acepta tráfico en el puerto NFS (2049) originado en las instancias web.



Utilidad de esta Arquitectura

Esta infraestructura está diseñada siguiendo las buenas prácticas de arquitectura en AWS (Well-Architected Framework):

- **Seguridad (DMZ):** Los servidores de aplicaciones y las bases de datos no tienen IPs públicas; están protegidos en subredes privadas. El único punto de entrada expuesto es el ALB.
- **Alta Disponibilidad:** Si una zona de disponibilidad de AWS sufre una caída, el ALB redirigirá el tráfico a la otra zona y el ASG repondrá las instancias caídas automáticamente.
- **Persistencia del Estado:** Al desacoplar los archivos adjuntos en EFS y los datos estructurados en RDS, las instancias EC2 se vuelven "inmutables" o stateless, facilitando el escalado horizontal continuo.

- **Cifrado y Gestión de Configuración:** Las contraseñas nunca viajan hardcodeadas en los scripts. Al arrancar la máquina, el script de User Data usa la AWS CLI para leer el secreto en caliente directamente desde Secrets Manager.

Postproceso a realizar (Acciones Manuales Obligatorias)

Dado que en el código de Terraform has configurado un dominio personalizado (`domain_name = "aws.juanjoguhu.net"`) y delegas la validación a un método manual, una vez que ejecutes `terraform apply`, debes realizar obligatoriamente los siguientes pasos:

1. Validar el Certificado SSL en AWS ACM

- Dirígete a la consola web de AWS y accede al servicio AWS Certificate Manager (ACM).
- Verás el certificado para tu dominio en estado "Pending validation" (Validación pendiente).
- Entra en los detalles del certificado y localiza la tabla de validación DNS. AWS te proporcionará un Nombre de CNAME y un Valor de CNAME.
- Ve al panel de gestión de DNS del proveedor donde tengas registrado tu dominio (ej. Cloudflare, GoDaddy, Nominalia, etc.).
- Crea un nuevo registro de tipo CNAME pegando esos valores exactos.
- Espera unos minutos. AWS comprobará el registro automáticamente y el estado del certificado cambiará a "Issued" (Emitido).

2. Apuntar tu dominio hacia el Application Load Balancer

- En la terminal donde ejecutaste Terraform, revisa los outputs y copia el valor devuelto en `alb_url` (o búscalo en la sección de EC2 -> Load Balancers en la consola de AWS). Es una DNS larga del estilo `tfaws-alb-XXXXXX.us-east-1.elb.amazonaws.com`.
- Vuelve al panel de gestión de DNS de tu proveedor de dominio.
- Crea un registro de tipo CNAME para tu subdominio (por ejemplo, `aws` si deseas mapear `aws.juanjoguhu.net`) apuntando directamente a la URL larga del ALB que acabas de copiar.
- Una vez propagados los cambios de DNS, cualquier petición que entre por HTTP (puerto 80) al dominio será redirigida por el listener del ALB de forma segura mediante un código 301 hacia HTTPS (puerto 443), sirviendo tu aplicación web Tomcat bajo una conexión cifrada y segura.

Esquema de la arquitectura desplegada

